Rapport de projet

Sécurisation d'une application web

reverse proxy avec WAF et durcissement HTTP

Réalisé par : Fatou Gueye
Master 1 Sécurité des Systèmes d'Information
École Supérieure Polytechnique de Dakar
Enseignant : Dr. Mouhamadane Diop

Année : 2026

Sommaire

1. Introduction et objectifs 3
1.1 Stack technique utilisée 3
1.2 Architecture 3
2. Mise en place du reverse proxy 4
2.1 Création du dossier de travail 4
2.2 Lancement des conteneurs 4
2.3 Vérification du reverse proxy 4
3. Tests des attaques OWASP Top 10 5
3.1 Vérification préliminaire 5
3.2 Les cinq tests de référence 5
3.3 Résultats 7
3.4 Preuve par l'absence : désactiver le WAF 7
4. Réglage fin du WAF : niveaux de paranoïa 10
4.1 PARANOIA=1 (niveau par défaut) 10
4.2 PARANOIA=2 (niveau retenu) 10
4.3 PARANOIA=3 11
4.4 PARANOIA=4 et faux positifs 11
4.5 Synthèse des niveaux 11
5. Durcissement HTTP : en-têtes, rate limiting et HTTPS 12
5.1 En-têtes de sécurité 12
5.2 Rate limiting anti brute-force 12
5.3 Activation HTTPS 13
6. Tester les limites du WAF : contournement par fragmentation 13
6.1 Fragmentation sur deux paramètres 14
6.2 Fragmentation de la balise elle-même 14
7. Fail2ban : bannissement automatique 14
7.1 Démarrage du service 15
7.2 Vérification de la jail 15
7.3 Déclenchement réel du bannissement 15
8. Validation externe : Nmap et OWASP ZAP 15
8.1 Scan Nmap 15
8.2 Scan OWASP ZAP : avec WAF actif 16
8.3 Scan OWASP ZAP : sans WAF (comparaison) 17
8.4 Réduction de la surface d'attaque 19
9. Limites identifiées 19
10. Conclusion 20

1. Introduction et objectifs

Ce projet consiste à mettre en place, de bout en bout, un Web Application Firewall (WAF) devant une application web volontairement vulnérable, OWASP Juice Shop, afin de mesurer concrètement l'efficacité d'une telle protection face aux attaques les plus courantes.

L'objectif n'était pas seulement de faire fonctionner un WAF, mais de le tester sérieusement : vérifier qu'il bloque bien les attaques du Top 10 OWASP, comprendre l'effet des différents niveaux de sensibilité (paranoia levels), identifier les faux positifs qu'il peut générer, tester ses limites face à des tentatives de contournement, et enfin valider l'ensemble depuis l'extérieur avec des outils indépendants (Nmap, OWASP ZAP).

L'ensemble de l'environnement a été construit avec Docker, ce qui permet de reproduire le projet à l'identique sur n'importe quelle machine en une seule commande.

1.1 Stack technique utilisée

Composant	Rôle
Docker / Docker Compose	Orchestration des conteneurs
Nginx + ModSecurity (OWASP CRS)	Reverse proxy et pare-feu applicatif (WAF)
OWASP Juice Shop	Application cible volontairement vulnérable
Fail2ban	Bannissement automatique des IP malveillantes
Nmap	Scan de ports et de vulnérabilités réseau, depuis Kali Linux
OWASP ZAP	Scan de sécurité applicative (baseline scan)
PowerShell / curl	Exécution et vérification des tests d'attaque

1.2 Architecture

Le trafic entrant passe d'abord par le reverse proxy Nginx couplé à ModSecurity, qui filtre les requêtes selon les règles OWASP CRS avant de les transmettre à Juice Shop. Les deux conteneurs communiquent sur un réseau Docker dédié ; aucune configuration Nginx n'a été écrite à la main, tout passe par les variables d'environnement de l'image `owasp/modsecurity-crs` (`BACKEND`, `PORT`, `PARANOIA`…).

Chemin d'une requête : Client (navigateur / curl / Nmap / ZAP) → Nginx + ModSecurity (OWASP CRS, ports 80/443) → Juice Shop (port 3000, interne).

2. Mise en place du reverse proxy

La première étape a consisté à créer le dossier de travail, puis à définir un `docker-compose.yml` regroupant Juice Shop et le reverse proxy WAF. Ce fichier est la pièce centrale du projet : il installe et configure les deux services en une seule commande.

2.1 Création du dossier de travail

```
PS C:\Users\HP\Documents> mkdir C:\waf-project


    Répertoire : C:\


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        16/08/2026     18:47                waf-project


PS C:\Users\HP\Documents> cd C:\waf-project
PS C:\waf-project>
```

Figure 1 : *Création du dossier `C:\waf-project` depuis PowerShell.*

2.2 Lancement des conteneurs

La commande `docker compose up -d` démarre les deux conteneurs (`nginx-waf` et `juiceshop`). Le `docker ps` qui suit confirme que les deux tournent correctement.

```
PS C:\waf-project> docker compose up -d
[+] Running 2/2
 ✔Container nginx-waf  Started                                                                                    1.9s
 ✔Container juiceshop  Started                                                                                    0.7s
PS C:\waf-project> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED          STATUS                            PORT
S                                  NAMES
6943e4bfd0f6   owasp/modsecurity-crs:nginx-alpine   "/docker-entrypoint.…"   19 seconds ago   Up 18 seconds (health: starting)  0.0.
0.0:80->8080/tcp, [::]:80->8080/tcp   nginx-waf
7e08ec42aa12   bkimminich/juice-shop               "/nodejs/bin/node /j…"   3 minutes ago    Up 19 seconds                     3000
/tcp                               juiceshop
PS C:\waf-project>
```

Figure 2 : *`docker compose up -d` puis `docker ps` : les deux conteneurs sont actifs (Up).*



Figure 3 : *Vue Docker Desktop confirmant que `juiceshop` et `nginx-waf` tournent.*

2.3 Vérification du reverse proxy

En ouvrant `http://localhost`, la page d'accueil de Juice Shop s'affiche normalement, ce qui prouve que Nginx redirige bien le trafic du port 80 vers le conteneur `juiceshop` (port 3000 interne), sans aucune ligne de configuration Nginx écrite manuellement.

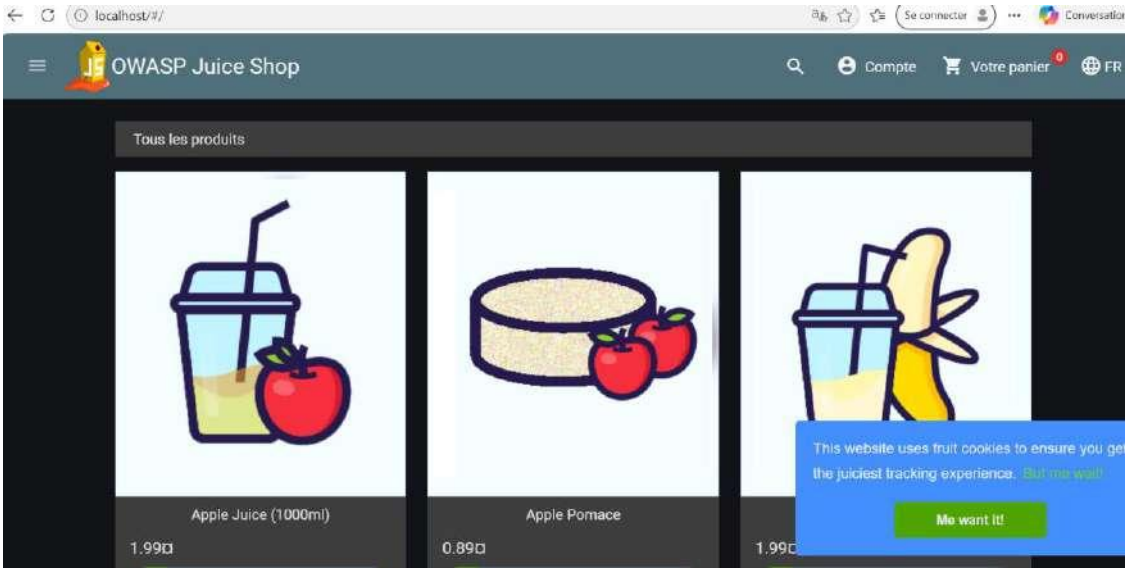


Figure 4 : *Page d'accueil de Juice Shop, accessible via le reverse proxy.*

3. Tests des attaques OWASP Top 10

Une fois le reverse proxy opérationnel, l'étape suivante a consisté à prouver que le WAF bloque réellement les attaques classiques. Chaque test a été rejoué avec `curl` directement sur le proxy.

3.1 Vérification préliminaire

Un premier test d'injection SQL a été effectué dès la mise en service, pour confirmer immédiatement que le WAF réagissait.

```
PS C:\waf-project> curl.exe -i "http://localhost/rest/products/search?q=1+OR+1=1"
HTTP/1.1 403 Forbidden
Server: nginx
Date: Sun, 16 Aug 2026 19:19:14 GMT
Content-Type: text/plain
Content-Length: 146
Connection: keep-alive
Access-Control-Allow-Origin: *
Access-Control-Max-Age: 3600
Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
Access-Control-Allow-Headers: *

<html>
<head><title>403 Forbidden</title></head>
<body>
<center><h1>403 Forbidden</h1></center>
<hr><center>nginx</center>
</body>
</html>
PS C:\waf-project>
```

Figure 5 : *Premier test d'injection SQL : 403 Forbidden.*

3.2 Les cinq tests de référence

Les cinq tests suivants ont ensuite été menés dans l'ordre, avec PARANOIA=2 (niveau retenu pour le projet, voir section 4) :

```
PS C:\waf-project> curl.exe -i "http://localhost/rest/products/search?q=1+OR+1=1"
HTTP/1.1 403 Forbidden
Server: nginx
Date: Sun, 16 Aug 2026 19:19:14 GMT
Content-Type: text/plain
Content-Length: 146
Connection: keep-alive
Access-Control-Allow-Origin: *
Access-Control-Max-Age: 3600
Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
Access-Control-Allow-Headers: *

<html>
<head><title>403 Forbidden</title></head>
<body>
<center><h1>403 Forbidden</h1></center>
<hr><center>nginx</center>
</body>
</html>
```

Figure 6 : *Test 1, injection SQL (`?q=1 OR 1=1`) : 403 Forbidden.*

```
PS C:\waf-project> curl.exe -i "http://localhost/?q=<script>alert(1)</script>"
HTTP/1.1 403 Forbidden
Server: nginx
Date: Wed, 19 Aug 2026 00:59:03 GMT
Content-Type: text/plain
Content-Length: 146
Connection: keep-alive
Access-Control-Allow-Origin: *
Access-Control-Max-Age: 3600
Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
Access-Control-Allow-Headers: *

<html>
<head><title>403 Forbidden</title></head>
<body>
<center><h1>403 Forbidden</h1></center>
<hr><center>nginx</center>
</body>
</html>
```

Figure 7 : *Test 2, XSS (`<script>alert(1)</script>`) : 403 Forbidden.*

```
PS C:\waf-project> curl.exe -i "http://localhost/?file=../../../etc/passwd"
HTTP/1.1 403 Forbidden
Server: nginx
Date: Wed, 19 Aug 2026 00:59:34 GMT
Content-Type: text/plain
Content-Length: 146
Connection: keep-alive
Access-Control-Allow-Origin: *
Access-Control-Max-Age: 3600
Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
Access-Control-Allow-Headers: *

<html>
<head><title>403 Forbidden</title></head>
<body>
<center><h1>403 Forbidden</h1></center>
<hr><center>nginx</center>
</body>
</html>
```

Figure 8 : *Test 3, LFI / traversée de répertoire (`../../../etc/passwd`) : 403 Forbidden.*

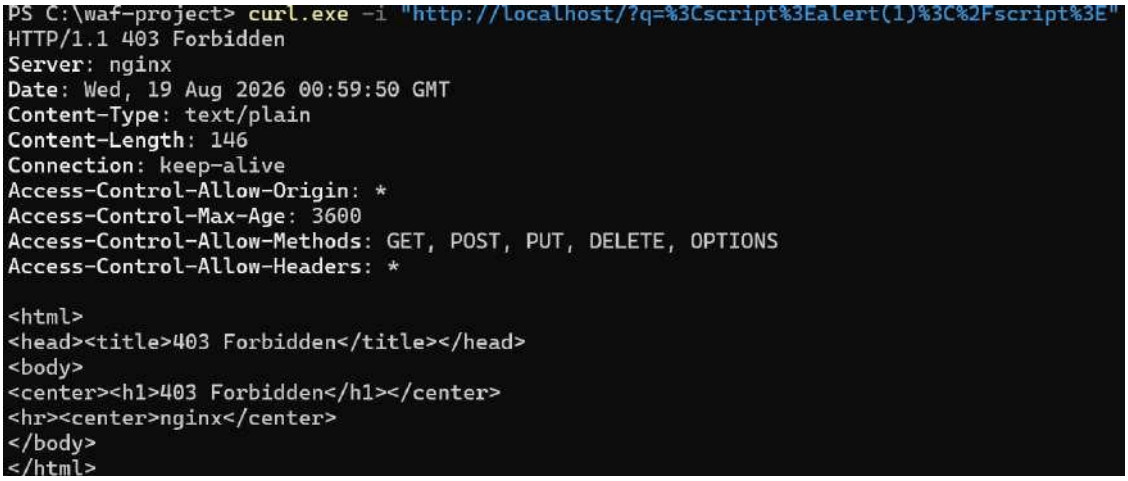


Figure 9 : *Test 4, XSS encodé en URL : 403 Forbidden ; le WAF décode la requête avant analyse.*

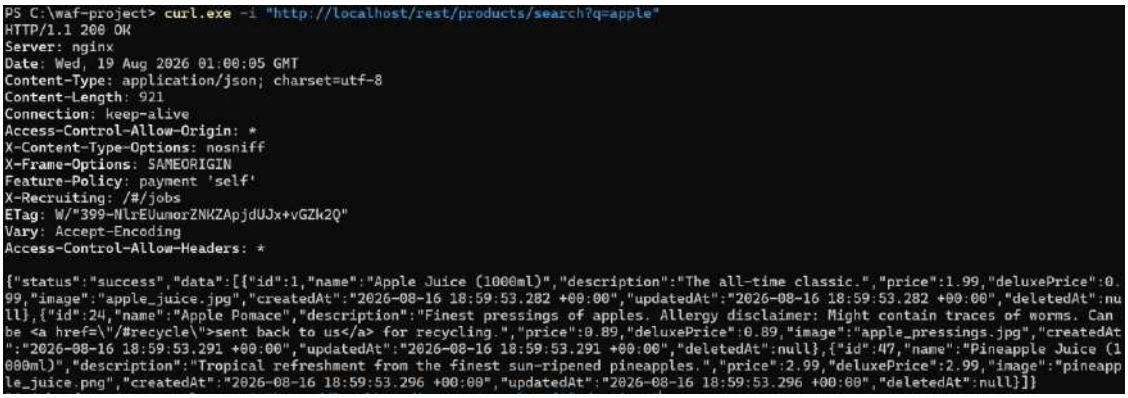


Figure 10 : *Test 5, recherche légitime (`?q=apple`) : 200 OK, aucun faux positif.*

3.3 Résultats

Test	Payload	Résultat
Injection SQL	`?q=1 OR 1=1`	403 Forbidden : bloqué
XSS	`?q=<script>alert(1)</script>`	403 Forbidden : bloqué
LFI	`?file=../../../etc/passwd`	403 Forbidden : bloqué
XSS encodé	`?q=%3Cscript%3E...`	403 Forbidden : bloqué
Requête légitime	`?q=apple`	200 OK : passe normalement

Taux de détection : 100 % sur les quatre attaques testées, sans faux positif sur le trafic légitime.

3.4 Preuve par l'absence : désactiver le WAF

Pour bien démontrer que c'est réellement le WAF qui bloque ces attaques (et non autre chose, comme une protection interne de Juice Shop), ModSecurity a été désactivé temporairement, et les cinq mêmes requêtes ont été rejouées.

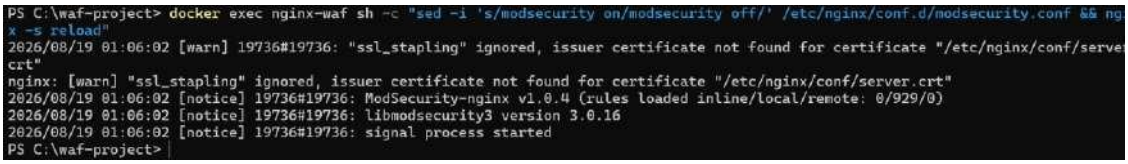


Figure 11 : *Désactivation de ModSecurity dans le conteneur* `nginx-waf`.

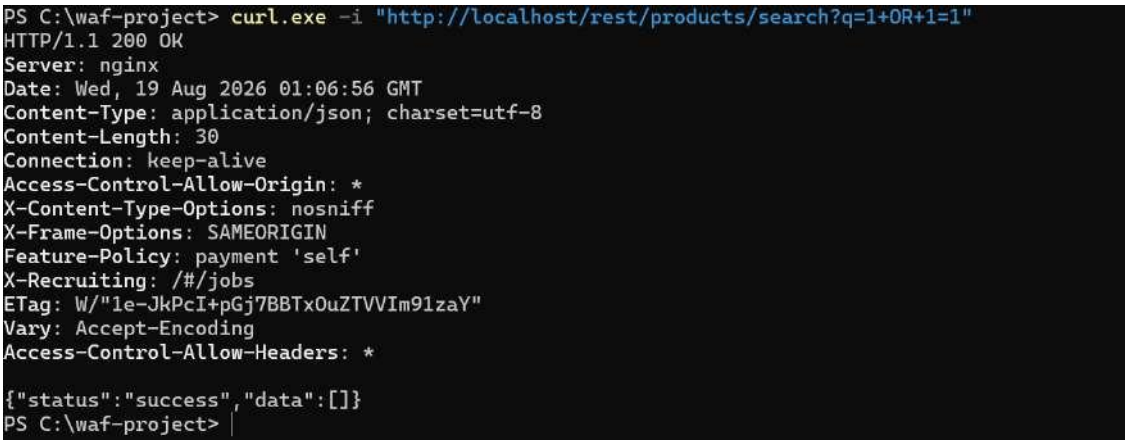


Figure 12 : *Sans WAF, l'injection SQL passe : 200 OK (au lieu de 403).*

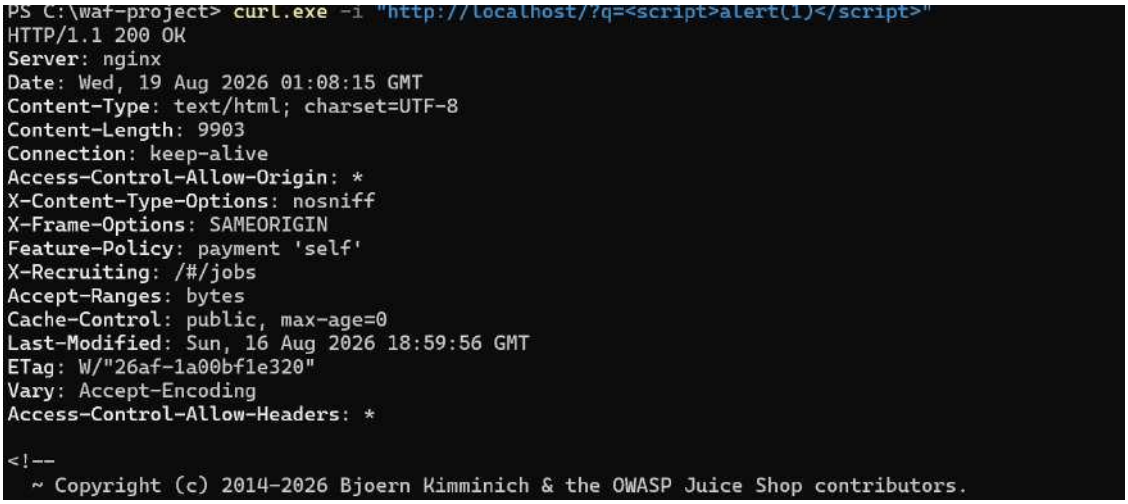


Figure 13 : *Sans WAF, le XSS passe : 200 OK, le payload est renvoyé tel quel par la page.*

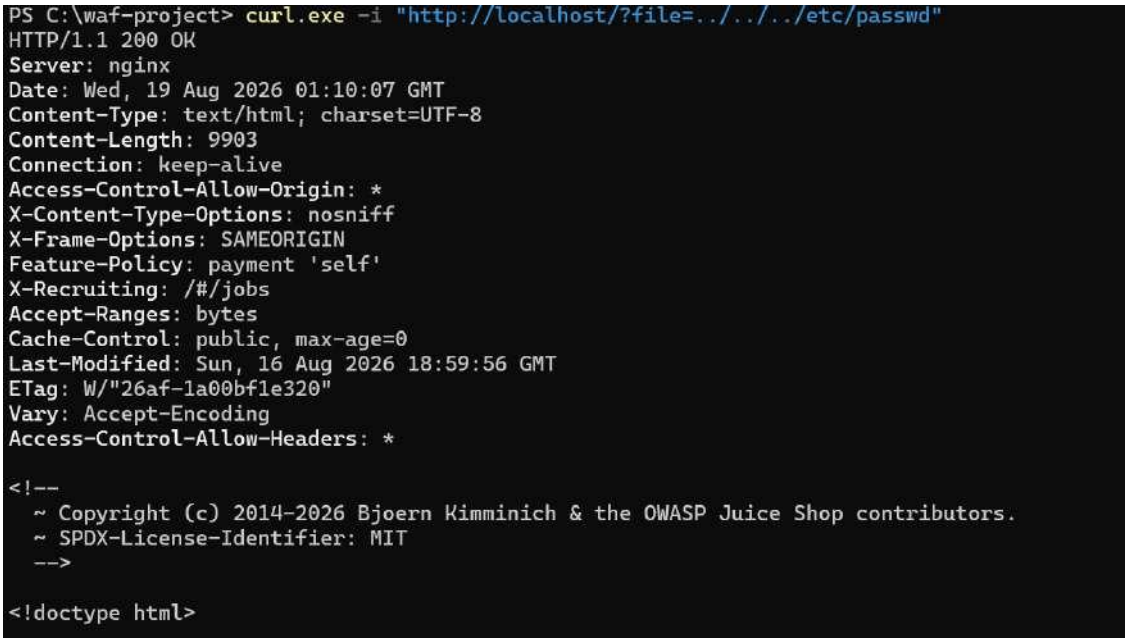


Figure 14 : *Sans WAF, la LFI passe : 200 OK.*

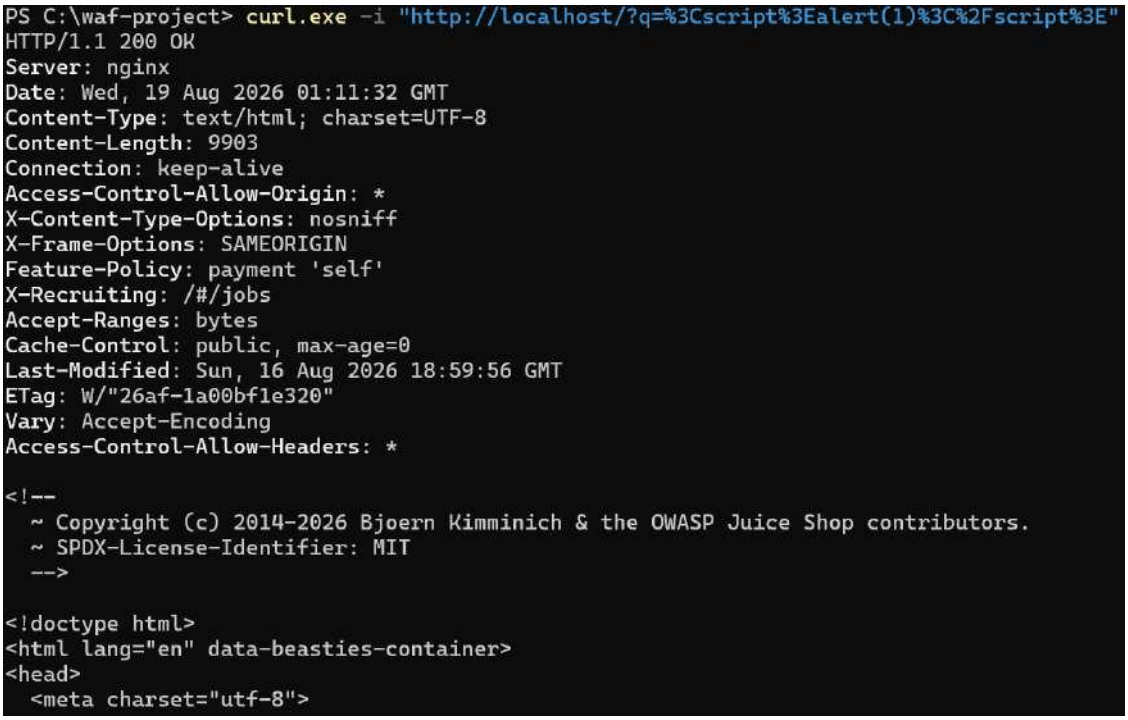


Figure 15 : *Sans WAF, le XSS encodé passe également : 200 OK.*



Figure 16 : *La requête légitime continue de fonctionner, comme attendu : 200 OK.*

Cette comparaison avant/après est la preuve la plus solide du rôle du WAF : sans lui, les quatre attaques passent sans aucune résistance. C'est cette différence qui justifie sa mise en place.

4. Réglage fin du WAF : niveaux de paranoïa

ModSecurity propose 4 niveaux de sensibilité (paranoia levels). Plus le niveau est élevé, plus la détection est fine, mais plus le risque de faux positifs augmente. Un script de test réutilisable (requête légitime, SQLi, XSS, LFI, User-Agent suspect) a été rejoué à chaque changement de niveau.

4.1 PARANOIA=1 (niveau par défaut)



Figure 17 : *À PARANOIA=1 : toutes les attaques évidentes sont bloquées (403), la requête normale passe (200).*

4.2 PARANOIA=2 (niveau retenu)



Figure 18 : *Passage à PARANOIA=2 et rejeu des tests : comportement identique, toutes les attaques bloquées.*

4.3 PARANOIA=3



Figure 19 : *Passage à PARANOIA=3 : détection comportementale renforcée, toujours aucun faux positif sur ces tests.*

4.4 PARANOIA=4 et faux positifs

À PARANOIA=4, deux tests supplémentaires ont été ajoutés pour vérifier l'impact sur du trafic parfaitement légitime : une recherche contenant une apostrophe (« l'orange ») et une requête POST JSON valide.

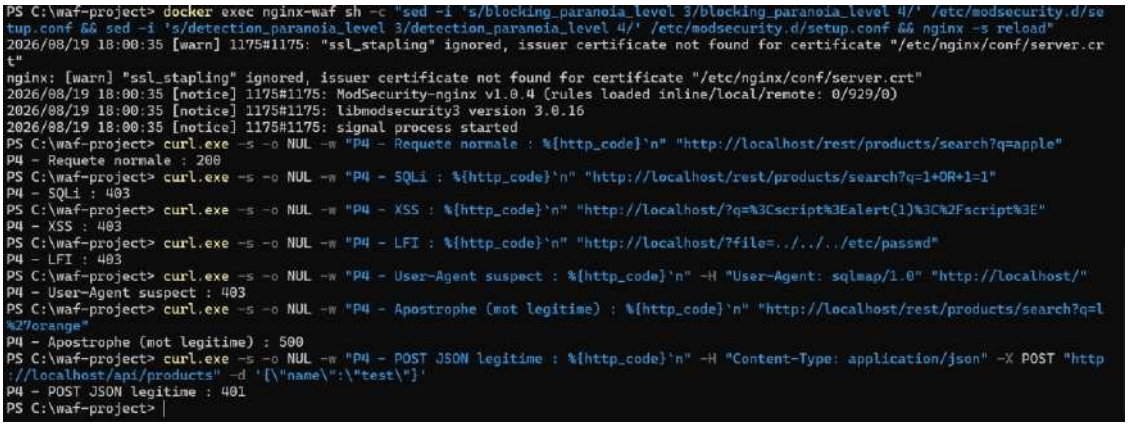


Figure 20 : *À PARANOIA=4 : l'apostrophe dans « l'orange » déclenche une erreur 500, et la requête POST JSON légitime est rejetée (401) au lieu de passer normalement.*

Ces deux résultats sont exactement l'argument qui justifie de ne pas monter le niveau de paranoïa au maximum par défaut : le gain de détection ne compense pas la dégradation de l'expérience utilisateur sur du trafic normal.

4.5 Synthèse des niveaux

Niveau	Détection	Faux positifs observés	Verdict
PARANOIA=1	Attaques évidentes	Aucun	Trop permissif pour ce projet
PARANOIA=2	+ encodages avancés	Aucun	Retenu : bon compromis
PARANOIA=3	+ détection comportementale	Aucun sur nos tests	Envisageable pour sites sensibles
PARANOIA=4	Maximum	Apostrophe (500), POST JSON (401)	Déconseillé en usage normal

Le niveau PARANOIA=2 a été conservé comme configuration finale du projet.

5. Durcissement HTTP : en-têtes, rate limiting et HTTPS

5.1 En-têtes de sécurité

Trois en-têtes de sécurité ont été ajoutés à la configuration Nginx : Strict-Transport-Security (HSTS), Content-Security-Policy (CSP) et Referrer-Policy.

```
PS C:\waf-project> curl.exe -I http://localhost
HTTP/1.1 200 OK
Server: nginx
Date: Sat, 29 Aug 2026 11:53:59 GMT
Content-Type: text/html; charset=UTF-8
Content-Length: 9903
Connection: keep-alive
Access-Control-Allow-Origin: *
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Feature-Policy: payment 'self'
X-Recruiting: /#/jobs
Accept-Ranges: bytes
Cache-Control: public, max-age=0
Last-Modified: Sat, 29 Aug 2026 11:23:40 GMT
ETag: W/"26af-1a04d42d48c"
Vary: Accept-Encoding
Access-Control-Allow-Headers: *
Strict-Transport-Security: max-age=31536000
Content-Security-Policy: default-src 'self'
Referrer-Policy: strict-origin-when-cross-origin

PS C:\waf-project>
```

Figure 21 : *`curl -I http://localhost` : les trois en-têtes sont bien présents dans la réponse.*

5.2 Rate limiting anti brute-force

Un rate limiting a été mis en place sur l'endpoint `/rest/user/login` (5 requêtes par minute, burst de 3). Un test de 10 tentatives de connexion successives a été lancé pour vérifier son déclenchement.

```
PS C:\waf-project> for ($i=1; $i -le 10; $i++) { curl.exe -s -o NUL -w "Tentative $i : %{http_code}\n" -X POST http://localhost/rest/
user/login -H "Content-Type: application/json" -d '{\"email\":\"test@test.com\",\"password\":\"wrong\"}' }
Tentative 1 : 401
Tentative 2 : 401
Tentative 3 : 401
Tentative 4 : 401
Tentative 5 : 429
Tentative 6 : 429
Tentative 7 : 429
Tentative 8 : 429
Tentative 9 : 429
Tentative 10 : 429
PS C:\waf-project>
```

Figure 22 : *Les 4 premières tentatives renvoient 401 (identifiants invalides), puis, à partir de la 5e, le rate limiting prend le relais avec des réponses 429 (Too Many Requests).*

5.3 Activation HTTPS

Un certificat auto-signé a été généré et le port 443 a été exposé. La page se charge normalement en HTTPS (l'option `-k` de curl ignore l'avertissement de certificat auto-signé, normal en environnement de test), avec les en-têtes de sécurité toujours présents.

```
PS C:\waf-project> curl.exe -k -i https://localhost
HTTP/1.1 200 OK
Server: nginx
Date: Sat, 29 Aug 2026 12:47:10 GMT
Content-Type: text/html; charset=UTF-8
Content-Length: 9903
Connection: keep-alive
Access-Control-Allow-Origin: *
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Feature-Policy: payment 'self'
X-Recruiting: /#/jobs
Accept-Ranges: bytes
Cache-Control: public, max-age=0
Last-Modified: Sat, 29 Aug 2026 11:23:40 GMT
ETag: W/"26af-1a04d42d48c"
Vary: Accept-Encoding
Access-Control-Allow-Headers: *
Strict-Transport-Security: max-age=31536000
Content-Security-Policy: default-src 'self'
Referrer-Policy: strict-origin-when-cross-origin

<!--
  ~ Copyright (c) 2014-2026 Bjoern Kimminich & the OWASP Juice Shop contributors.
  ~ SPDX-License-Identifier: MIT
  -->

<!doctype html>
<html lang="en" data-beasties-container>
<head>
  <meta charset="utf-8">
  <title>OWASP Juice Shop</title>
  <meta name="description" content="Probably the most modern and sophisticated insecure web application">
```

Figure 23 : *`curl -k -i https://localhost` : 200 OK, en-têtes de sécurité toujours actifs.*

Le WAF a ensuite été retesté en HTTPS pour confirmer qu'il protège aussi ce canal.

```
PS C:\waf-project> curl.exe -k -s -o NUL -w "HTTPS SQLi : %{http_code}`n" "https://localhost/rest/products/search?q=1+OR+1=1"
HTTPS SQLi : 403
PS C:\waf-project> curl.exe -k -s -o NUL -w "HTTPS Normal : %{http_code}`n" "https://localhost/rest/products/search?q=apple"
HTTPS Normal : 200
PS C:\waf-project> curl.exe -k -s -o NUL -w "HTTPS - XSS : %{http_code}`n" "https://localhost/?q=%3Cscript%3Ealert(1)%3C%2Fscript%3E"

HTTPS - XSS : 403
PS C:\waf-project>
```

Figure 24 : *En HTTPS : le SQLi et le XSS sont bloqués (403), la requête normale passe (200).*

6. Tester les limites du WAF : contournement par fragmentation

Pour montrer une compréhension réelle des limites du WAF (et pas seulement de ses succès), deux techniques de contournement par fragmentation ont été testées : répartir un payload sur deux paramètres différents, et couper une balise HTML en deux morceaux.

6.1 Fragmentation sur deux paramètres

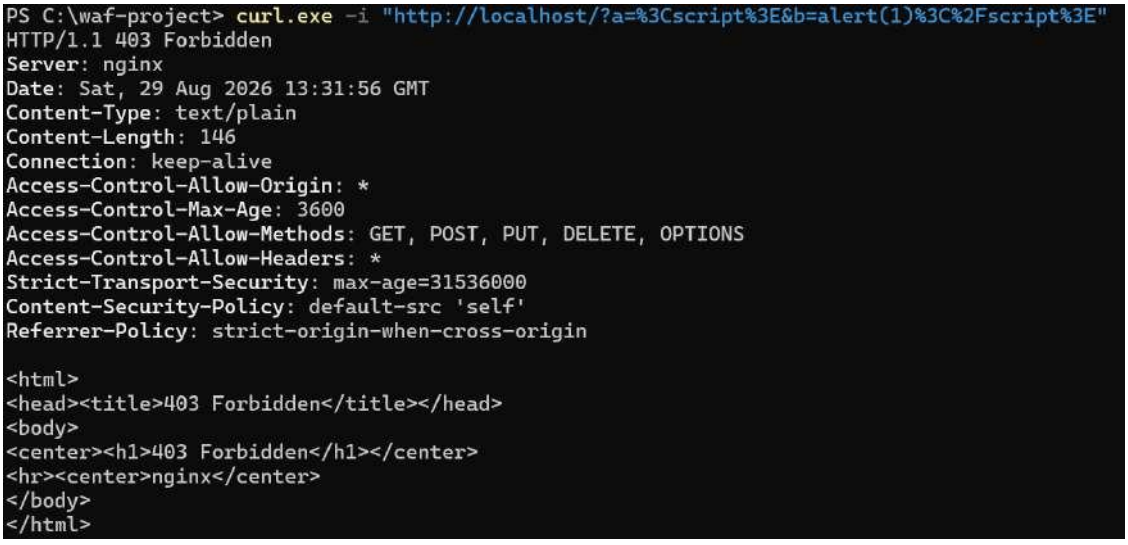


Figure 25 : *Un payload XSS réparti sur deux paramètres (`a=` et `b=`) : toujours bloqué (403), grâce au score cumulé des règles CRS.*

6.2 Fragmentation de la balise elle-même

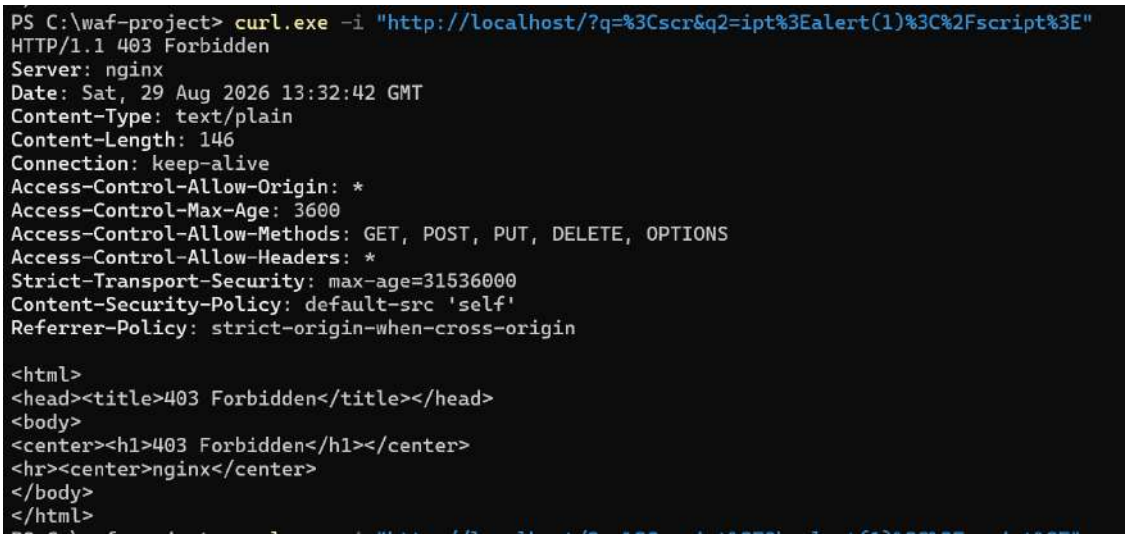


Figure 26 : *La balise `<script>` coupée en deux paramètres distincts : bloquée également (403).*

Il est important de rester honnête sur la portée de ce résultat : ces deux tests montrent que ces deux cas précis sont détectés, mais ne prouvent pas que toute forme de fragmentation l'est. Une fragmentation plus fine, répartie sur trois paramètres ou plus, pourrait théoriquement passer sous le score cumulé de détection. C'est une limite réelle du WAF, pas un aspect à passer sous silence.

7. Fail2ban : bannissement automatique

Fail2ban a été ajouté au `docker-compose.yml` pour surveiller les logs du WAF et bannir automatiquement toute IP générant plusieurs réponses 403 en peu de temps (3 échecs en 60 secondes, bannissement de 5 minutes).

7.1 Démarrage du service



Figure 27 : *`docker compose up -d fail2ban` : le conteneur démarre correctement.*

7.2 Vérification de la jail

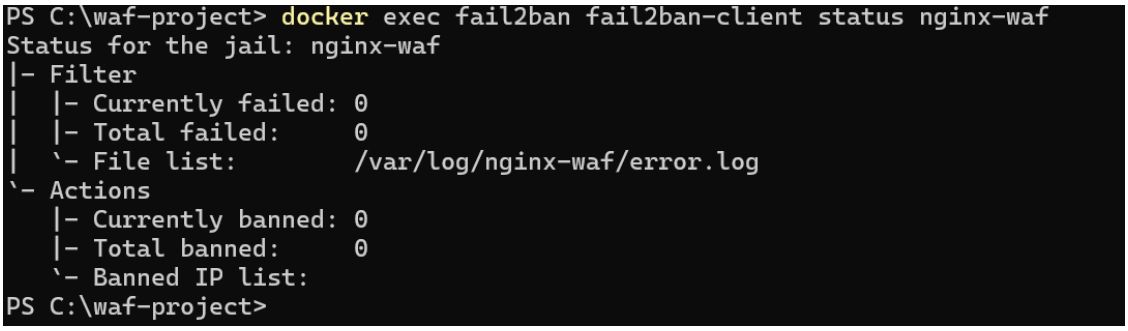


Figure 28 : *La jail `nginx-waf` est active, 0 IP bannie au départ (état initial).*

7.3 Déclenchement réel du bannissement

Pour vérifier que la jail fonctionne réellement (et pas seulement qu'elle est « active » sur le papier), plusieurs requêtes d'injection SQL ont été envoyées volontairement pour déclencher des 403 répétés.

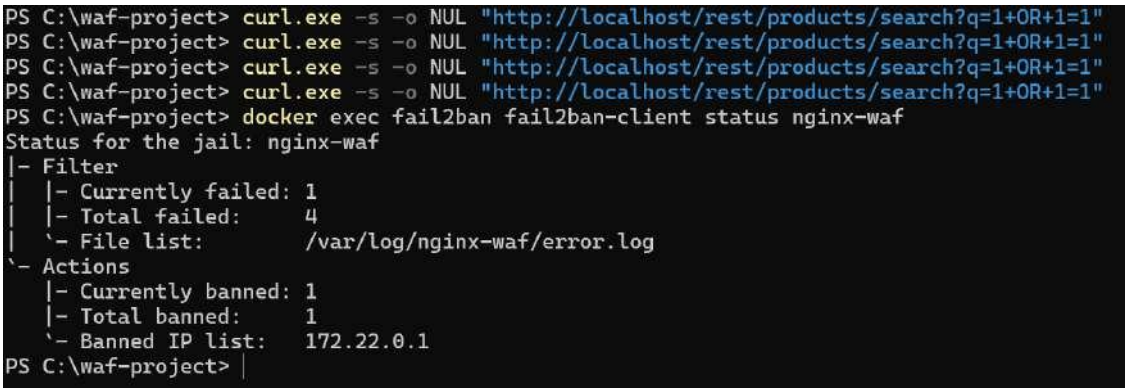


Figure 29 : *Après plusieurs tentatives de SQLi, Fail2ban détecte 4 échecs et bannit automatiquement l'IP 172.22.0.1.*

Ce test confirme que la défense en profondeur fonctionne à deux niveaux : le WAF bloque la requête (403), puis Fail2ban bannit la source si elle insiste.

8. Validation externe : Nmap et OWASP ZAP

La dernière étape a consisté à valider le dispositif depuis l'extérieur, avec des outils indépendants du projet, lancés depuis une machine Kali Linux.

8.1 Scan Nmap

Un scan de ports et un scan de vulnérabilités ont été lancés contre l'adresse IP de la machine hébergeant le projet.

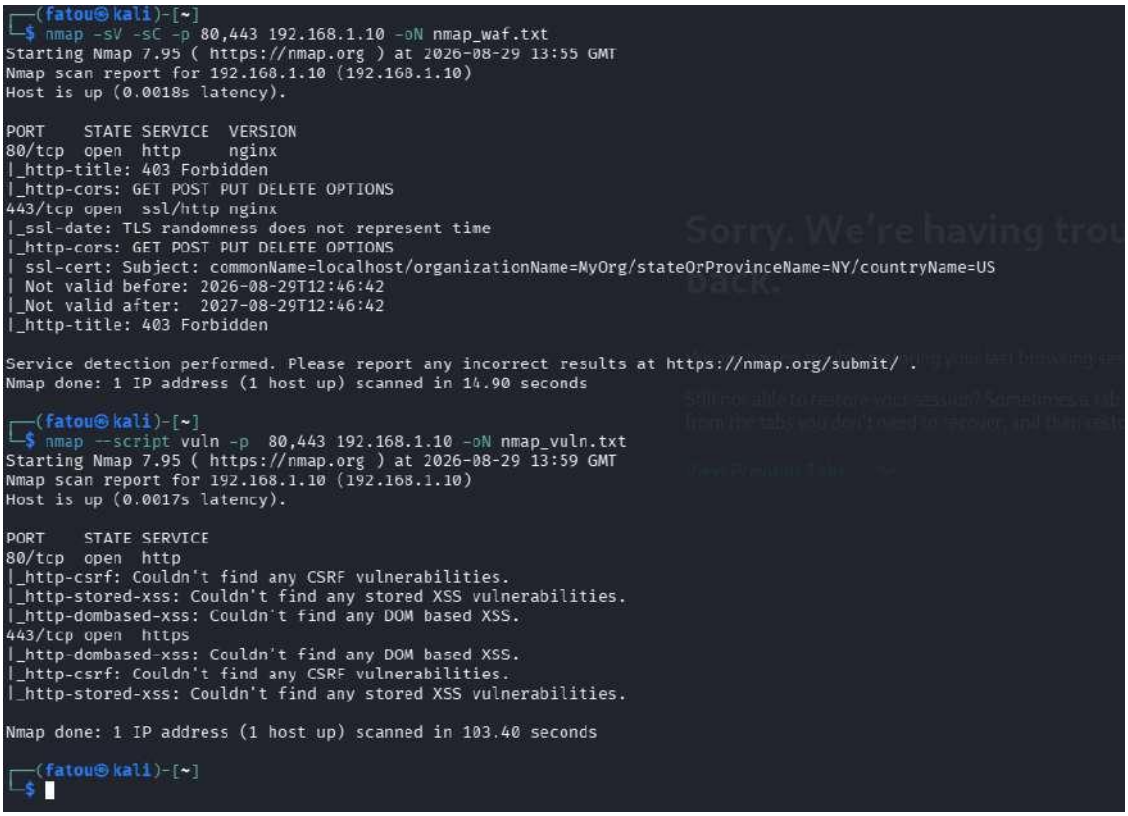


Figure 30 : *Nmap : seuls les ports 80 et 443 apparaissent ouverts, aucune vulnérabilité CSRF ou XSS détectée par les scripts* `vuln`.

Le résultat confirme que la surface réseau exposée est minimale et que la version exacte de Nginx n'est pas exploitable pour une reconnaissance poussée.

8.2 Scan OWASP ZAP : avec WAF actif

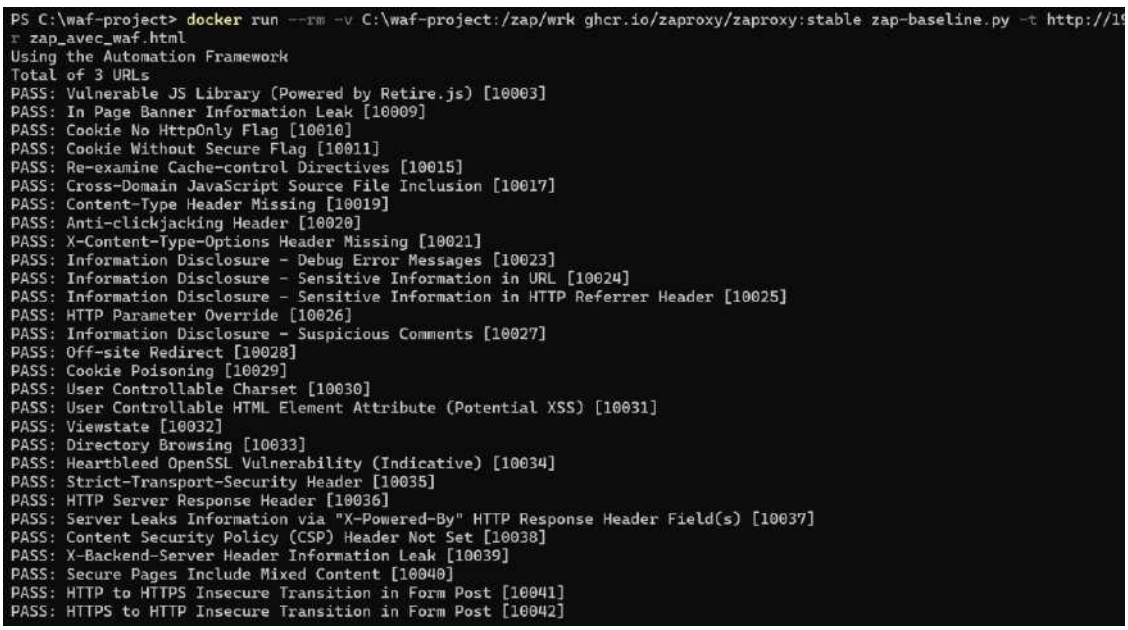


Figure 31 : *Lancement du scan ZAP baseline avec WAF actif (*`zap_avec_waf.html`*) : 3 URLs explorées seulement.*

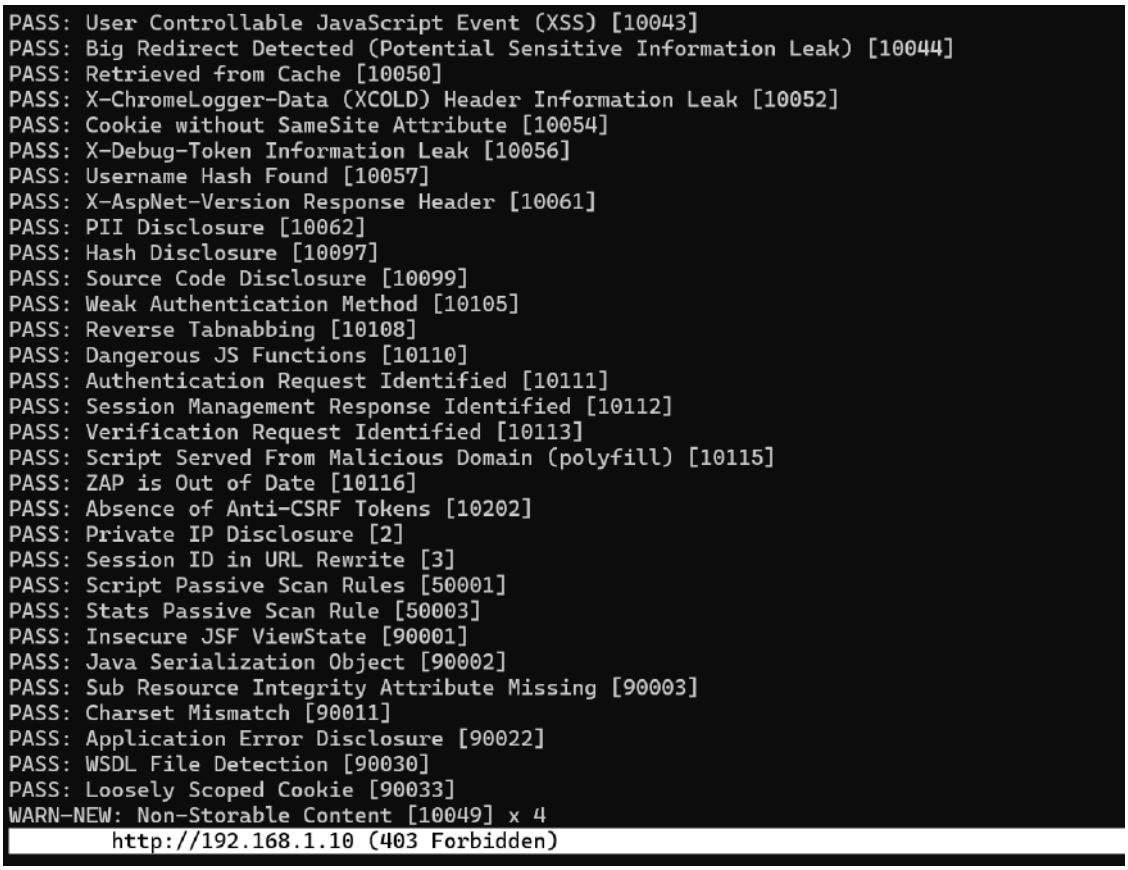


Figure 32 : *Suite du scan : la grande majorité des contrôles passent (PASS).*

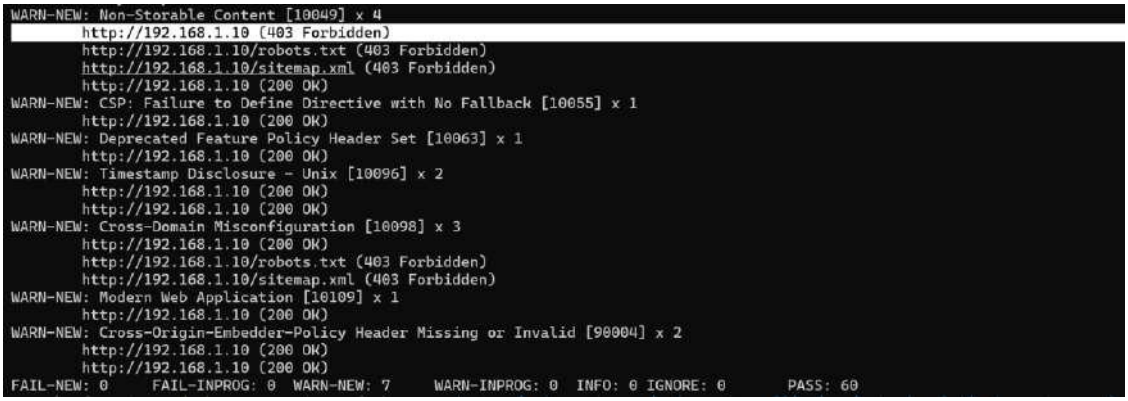


Figure 33 : *Résultat final avec WAF : 0 FAIL, 7 WARN, 60 PASS. Même `robots.txt` et `sitemap.xml` sont bloqués (403) par le WAF.*

8.3 Scan OWASP ZAP : sans WAF (comparaison)

Le WAF a été désactivé temporairement pour établir une comparaison objective, avec un scan identique lancé sur la même cible.

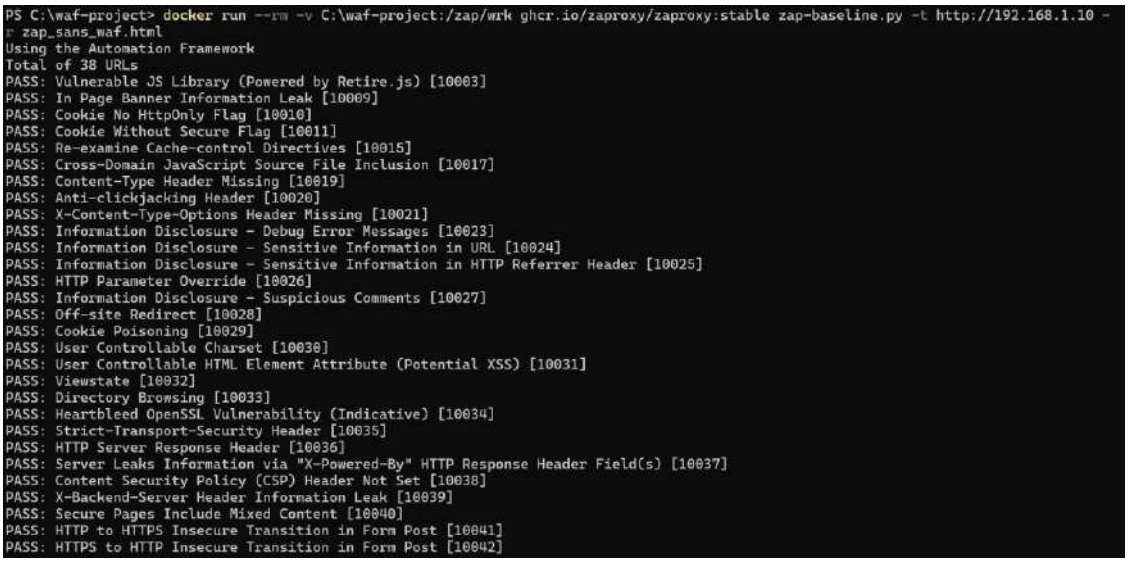


Figure 34 : *Sans WAF, ZAP explore 38 URLs (contre 3 avec le WAF actif) : la surface d'attaque exposée est nettement plus grande.*

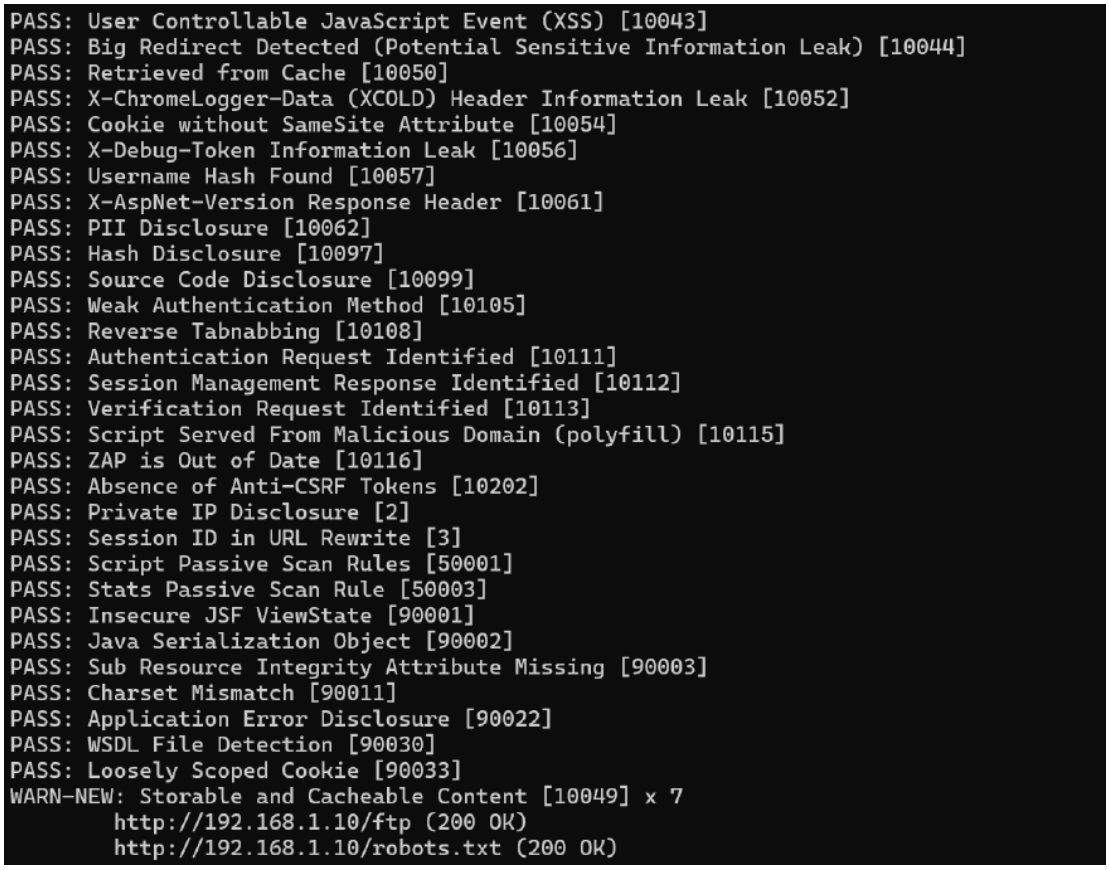


Figure 35 : *De nombreux fichiers internes (chunks JS, `styles.css`, `sitemap.xml`, `robots.txt`) sont accessibles en 200 OK.*

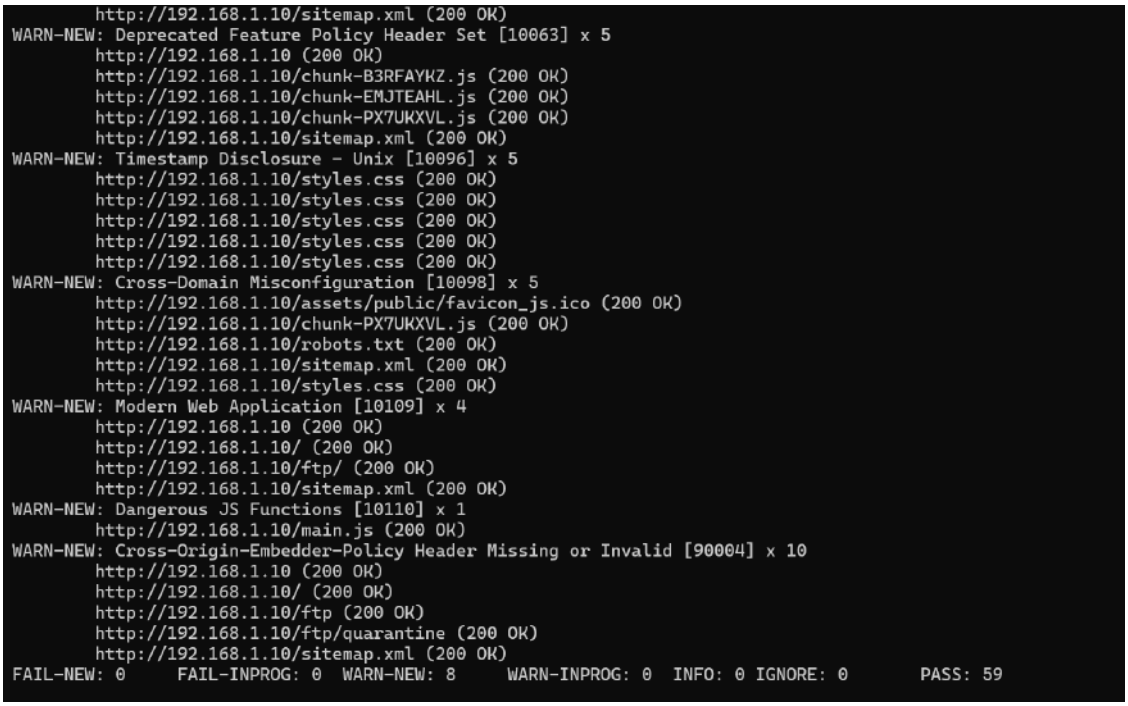


Figure 36 : *Résultat final sans WAF : 0 FAIL, 8 WARN, 59 PASS, mais sur une surface bien plus large (38 URLs contre 3).*

8.4 Réduction de la surface d'attaque

La comparaison des deux scans donne le résultat le plus parlant du projet : le nombre d'URLs explorables par ZAP passe de 38 (sans WAF) à 3 (avec WAF), soit une réduction d'environ 92 % de la surface d'attaque exposée à un scanner externe.

Important : le WAF a été réactivé immédiatement après cette comparaison, avec une vérification explicite par `curl` confirmant le retour du blocage sur l'injection SQL, afin de ne pas laisser la cible exposée plus longtemps que nécessaire.

9. Limites identifiées

- Le WAF détecte des attaques par signature (règles OWASP CRS) : il ne protège pas contre les failles logiques métier (abus d'une fonctionnalité légitime de l'application) ni contre des attaques zero-day non couvertes par les règles.
- Les tests de fragmentation menés (2 paramètres, balise coupée en deux) ont été bloqués, mais ne prouvent pas que toute fragmentation est détectée : une répartition plus fine sur 3 paramètres ou plus pourrait théoriquement passer sous le score cumulé.
- Le choix du niveau PARANOIA=2 est un compromis pragmatique entre détection et faux positifs, pas une garantie de sécurité absolue : PARANOIA=4 montre qu'une détection plus agressive dégrade l'expérience sur du trafic légitime.
- La validation Nmap/ZAP a été faite dans un environnement de test local ; les résultats (92 % de réduction de surface) sont représentatifs de cette configuration précise et ne se généralisent pas automatiquement à toute application.

10. Conclusion

Ce projet a permis de construire une chaîne de défense complète devant une application vulnérable : un WAF filtrant 100 % des attaques OWASP Top 10 testées sans faux positif, un réglage de sensibilité justifié par des tests concrets, un durcissement HTTP (en-têtes, rate limiting, HTTPS), un bannissement automatique via Fail2ban, et une validation indépendante montrant une réduction mesurée de 92 % de la surface d'attaque exposée.

Au-delà des résultats positifs, le projet a aussi permis d'identifier honnêtement les limites de ce type de protection : un WAF réduit fortement le risque, mais ne le supprime pas, et doit être vu comme une couche de défense en profondeur parmi d'autres, pas comme une solution unique.